sportsdataverse :: CHEAT SHEET

The SportsDataverse's Node.js / TypeScript client (npm `sportsdataverse`) — sister to sportsdataverse-py and the R packages (hoopR, wehoop, cfbfastR, fastRhockey, baseballr). The cross-league ESPN client (126 endpoints × 30 leagues), 1,060 native / provider wrappers in 27 families, and 323 release-dataset loaders, with a tidy `{ parsed: true }` layer and one error vocabulary. **Names are sportsdataverse-py's.** Page 1 of 4: install, namespaces, parsers, errors.

v4.0.0

Node ≥ 20.18.1 · ESM · TypeScript
js.sportsdataverse.org

GET STARTED

```
npm install sportsdataverse
```

ESM-only, ships `.d.ts` typings. Optional peers: `impit` (browser TLS: stats.nba.com, 247Sports, KenPom) and `playwright` (NFL Pro login).

```
import sdv from "sportsdataverse";  // ESM
// CommonJS: require() on Node 20.19+ /
// 22.12+, or a dynamic import
const sdv = (await import(
  "sportsdataverse")).default;
```

QUICK START

```
const board = await sdv.nba
  .espnNbaScoreboard({ parsed: true });
```

```
const box = await sdv.nba.espnNbaSummary({
  event_id: 401584793, parsed: true,
  section: "boxscore_player" });
```

```
const pbp = await sdv.cfb.loadCfbPbp({
  seasons: [2024],
  columns: ["game_id", "text", "EPA"] });
```

Every wrapper is `async` and takes one params object; snake_case and camelCase keys are interchangeable: `{ team_id: 12 }` ≡ `{ teamId: 12 }`.

V4 NAMES = SDV-PY'S

An ESPN *athlete* is a *player*, an *event* a *game*, an *event_competitor* a *game_team*: `espnNbaPlayerGamelog`, `espnNflGameTeamRoster`. Every pre-v4 name still works and warns once (`SDV_DEPRECATED_NAME`). Each wrapper answers to snake (`espn_nfl_team_roster`) and camel (`espnNflTeamRoster`).

NAMESPACE LAYOUT

Namespace	Contents
`sdv.nba .wnba .mbb .wbb .cfb .nfl .mlb .nhl`	ESPN wrappers + native APIs + `load*` + legacy `get*` (page 2)
`sdv.nbagl .mch .wch .college_baseball .college_softball .ufl .xfl .cfl`	ESPN-only leagues
`sdv.soccer` + `epl laliga bundesliga seriea ligue1 mls ligamx ucl uel nwsl wwc wc`	league-param catch-all + 12 curated aliases (`mls`, `nwsl` also carry their stats APIs)
`sdv.cricket`	ESPN cricket + in-play win probability
`sdv.hockeytech .pwhl`	20 HockeyTech leagues · PWHL loaders
`sdv.odds .sports247 .on3 .cbs .fox .yahoo .torvik .asa`	provider namespaces (page 4)
`sdv.ncaa` · `sdv.tennis` · `sdv.recruiting`	legacy / deprecated services (page 2)

43 namespaces on the default export; `await functionCount("nba")` → 348.

MULTI-LEAGUE SPORTS

```
await sdv.soccer.espnSoccerScoreboard(
  { league: "eng.1" });  // Premier League
await sdv.laliga.espnLaligaScoreboard({});
```

`soccer` / `cricket` take a `league` slug; the curated aliases pin it for you.

DISCOVERY

```
await listFunctions("nba",
  { search: "roster" });
await findTeam("Celtics", "nba");
```

Also `functionCount`, `findAthlete(name, league)`, `findEvent(date, league, { home, away })` (snake twins too).

TIDY ROWS: { PARSED: TRUE }

Every wrapper returns the **raw** payload by default (`unknown` in TypeScript). `{ parsed: true }` runs the registered parser → flat, snake_cased row objects (py's `return_parsed=True`), typed (`MlbTeamRosterRow[]`, …) where the parser-parity harness verified the endpoint on a real capture.

```
const rows: MlbTeamRosterRow[] =
  await sdv.mlb.mlbTeamRoster(
    { team_id: 147, parsed: true });
```

Summary dispatcher (21 sections)

Sections: `boxscore_player boxscore_team plays winprobability leaders game_info officials header season_series against_the_spread standings broadcasts format pickcenter odds article injuries news drives drive_plays scoring_plays`. Omit `section` → all 21.

IDS ARE STRINGS

Integer id columns are decimal strings on every surface (`"401584793"`, exact past 2^{53}): compare and join ids as strings.

BROWSER-SAFE PARSERS

```
import { parseEndpoint }
  from "sportsdataverse/parsers";
```

No node-only HTTP deps; powers the docs playground. Also `normalize`, `PARSERS`, `parserFor`.

ERRORS (ALL `SDVERROR`)

Error	Means
`NoDataError`	fetch worked, nothing there (404, ESPN `{code: 404}`)
`AssetFetchError`	fetch failed (403 / 429 / 5xx after retries, network, empty body): answer unknown
`InvalidParameterError`	400 / 422; never retried
`SeasonNotFoundError`	season outside the source
`TransportUnavailableError`	optional peer missing

A failed fetch never comes back as empty data. An empty payload parses to `[]`.

CONFIGURE, TRANSPORT, AUTH

```
import { configure, bearerAuth }
  from "sportsdataverse";
configure({ retries: 5, timeoutMs: 60_000,
  auth: { pff_api: bearerAuth(KEY) } });
```

Per family (`site_v2`, `mlb`, `nba_stats`, …) with a `default`: `axiosTransport`, `createImpersonatingTransport`; auth `bearerAuth headerAuth queryAuth tokenAuth sessionAuth`; `getConfig` / `resetConfig`.

PIPE WITH TIDY.JS

```
import { tidy } from "sportsdataverse";
tidy.tidy(rows, tidy.groupBy("team",
  tidy.summarize({ n: tidy.n() })));
```

ECOSYSTEM

Python: sportsdataverse-py (same names). R: hoopR wehoop cfbfastR fastRhockey baseballr. Plots: @sportsdataverse/sdvplot. Docs, tutorials, playground, llms.txt: **js.sportsdataverse.org**.

Legacy services :: sdv.<league>.get*

Pre-3.0 hand-written methods, kept for back-compat. Since 4.0.0 they fetch through the shared request layer (https, retries, timeout, typed errors); new code should prefer the generated `espn_*` wrappers (page 3)

8 LEAGUE SERVICES · COMMON SHAPE

GAME ENDPOINTS

On `sdv.cfb .mbb .mlb .nba .nfl .nhl .wbb .wnba` — all take an ESPN game `id`:

Function	Returns / notes
`getPlayByPlay(id)`	teams, plays, competitions, season, boxScore subset
`getBoxScore(id)`	gamepackage boxscore (+ `id`)
`getSummary(id)`	full ESPN summary subset
`getPicks(id)`	pickcenter; cfb mbb mlb nba nfl nhl only

```
const pbp = await sdv.wnba
  .getPlayByPlay(401244185);
const box = await sdv.cfb
  .getBoxScore(401628334);
```

Source: `cdn.espn.com/core/<sport>/…` game-package feeds. `getPlayByPlay` returns `{teams, id, plays, competitions, season, boxScore, seasonSeries, standings}`.

TEAMS

Function	Args
`getTeamList()`	colleges: `{group}` (cfb 80, mbb/wbb 50)
`getTeamInfo(id)`	nfl: `{id}` object
`getTeamPlayers(id)`	roster; nfl: `{id}`

```
await sdv.mbb.getTeamList({ group: 50 });
await sdv.nba.getTeamInfo(13);
await sdv.nfl.getTeamPlayers({ id: 12 });
```

Default groups: 80 = FBS (cfb) · 50 = NCAA D1 (mbb / wbb).

SCHEDULES & STANDINGS

Function	Args
`getSchedule`	`{year, month, day}` + colleges: `groups, seasontype`
`getScoreboard`	same + `limit` (300–1000)
`getStandings`	`{year}` (defaults to now)
`getConferences`	`{year}`; cfb mbb wbb only
`getRankings`	`{year, week}`; cfb only
`getWeeklySchedule`	`{week, year, seasonType}`; nfl only

```
await sdv.cfb.getSchedule(
  { year: 2024, month: 9, day: 7,
    groups: 80, seasontype: 2 });
await sdv.nfl.getWeeklySchedule(
  { week: 1, year: 2024, seasonType: 2 });
await sdv.mbb.getStandings({ year: 2024 });
await sdv.wbb.getConferences({ year: 2024 });
await sdv.cfb.getRankings(
  { year: 2024, week: 5 });
```

AVAILABILITY MATRIX

Namespace	Method count
`sdv.cfb`	15 (+ rankings, recruiting)
`sdv.mbb`	14 (+ recruiting)
`sdv.nfl`	11 (+ weekly schedule)
`sdv.mlb sdv.nba sdv.nhl`	10 each
`sdv.wbb`	10 (conferences, no picks)
`sdv.wnba`	9 (no picks / conferences)

RECRUITING SCRAPERS · CFB + MBB

247SPORTS (LEGACY SCRAPE)

Function	Args
`getPlayerRankings`	`{year, page, group, position, state}` (+ cfb `rankingsType`)
`getSchoolRankings`	`(year, page = 1)`
`getSchoolCommits`	`(school, year)`

```
await sdv.cfb.getPlayerRankings(
  { year: 2025, position: "QB" });
await sdv.mbb.getSchoolCommits(
  "Duke", 2025);
```

HTML scrape via cheerio. Superseded by `sdv.sports247` (page 4); prefer it for new code.

TENNIS

```
await sdv.tennis.getScoreboard(
  { league: "atp", year: 2024,
    month: 7, day: 14 });
```

`league`: `"atp"` or `"wta"` (ESPN site v2 scoreboard).

MODERN REPLACEMENT

```
await sdv.sports247
  .sports247InstitutionRankings({
    sport_key: 1, year: 2025,
    parsed: true });
```

`sdv.sports247` (4.0.0): 12 API + 35 site-page wrappers; the free guest JWT is minted for you; needs `impit`. `sdv.recruiting` (25, caller JWT) is **deprecated**.

NCAA.COM · SDV.NCAA

GAMES (DATA.NCAA.COM)

Function	Purpose
`getInfo(game)`	gameInfo.json
`getBoxScore(game)`	boxscore.json
`getPlayByPlay(game)`	pbp.json
`getScoreboard({...})`	`{sport, division, year, month, day}`
`getRedirectUrl(url)`	resolve an ncaa.com game URL → casablanca id

```
const sb = await sdv.ncaa.getScoreboard({
  sport: "basketball-men", division: "d1",
  year: 2024, month: 3, day: 21 });
const url = sb.games[0].game.url;
const gid = await sdv.ncaa
  .getRedirectUrl(url);
const pbp = await sdv.ncaa.getPlayByPlay(gid);
```

STATS.NCAA.ORG RANKINGS

Function	Args
`getSports()`	—
`getSeasons(sport)`	sport code
`getDivisions(sport, season)`	—
`getSportDivisionData`	`(sport, season, division, type, gameHigh)`
`getPlayerData`	`(sport, season, division, rankingPeriod, gameHigh, category)`
`getTeamData`	same args as player

Every NCAA sport code (wvb, mih, wla, wsb, …). **Deprecated in 4.0.0**: these six stats.ncaa.org scrapers; stats.ncaa.org data lives in sportsdataverse-py and the hoopR / wehoop releases (`load*`).

ESPN cross-league :: 126 endpoints × 30 leagues

Same names as sportsdataverse-py's `espn_*` wrappers · keyless, no auth

PATTERN & SCOPES

One core, parameterized on ESPN `(sport, league)` slugs. Each league exposes `espn_<prefix>_<short>` / `espn<Prefix><Short>` for every short name in its scope set:

Scope	Leagues	Shorts
`universal`	all 30	117
`ncaa`	college leagues	+6
`football`	nfl · cfb	+2
`mlb`	mlb	+1

ncaa: `rankings season_recruits recruiting_years recruiting_players recruiting_rankings season_week_rankings` · football: `season_qbr season_qbr_week` · mlb: `player_hotzones`. CFB renames: `futures groups team_powerindex recruits week_rankings`.

```
await sdv.cfb.espnCfbRankings({});
await sdv.wnba.espnWnbaStandings(
  { season: 2024 });
await sdv.nfl.espnNflScoreboard(
  { week: 1, season_type: 2 });
```

ESPN-ONLY LEAGUES

```
await sdv.mch.espnMchScoreboard({});
await sdv.college_softball
  .espnCollegeSoftballScoreboard({});
await sdv.nbagl.espnNbaglStandings({});
```

LEAGUES (LEAGUES.YAML)

Sport	Prefixes
basketball	`nba wnba mbb wbb nbagl`
football	`nfl cfb ufl xfl cfl`
baseball	`mlb college_baseball college_softball`
hockey	`nhl mch wch`
soccer	`soccer` + 12 aliases
cricket	`cricket`

SITE V2 · 25 SHORTS

SCORES, TEAMS, NEWS

League: `scoreboard summary calendar news injuries transactions conferences statistics_league draft standings rankings`

Teams: `teams_site team team_roster team_schedule team_record team_depthcharts team_injuries team_transactions team_history team_news team_leaders`

Players: `player_info player_bio player_news`

WEB V3 · 5 · FITT V3 · 1

PLAYER STATS & FPI

`player_overview player_stats_v3 player_gamelog player_splits leaders` · `fpi` (ESPN Football / Basketball Power Index)

```
await sdv.nba.espnNbaPlayerGamelog(
  { athlete_id: 1966, season: 2024 });
await sdv.cfb.espnCfbFpi({ season: 2024 });
```

CDN.ESPN.COM · 5 SHORTS

GAME-PAGE DATA

`cdn_playbyplay cdn_boxscore cdn_schedule cdn_scoreboard cdn_rankings`: the espn.com page JSON, on the leagues that serve it.

SUMMARY SECTIONS

```
await sdv.cfb.espnCfbSummary({
  event_id: 401628334, parsed: true,
  section: "drive_plays" });
```

`drives drive_plays scoring_plays` populate for NFL / CFB; other sports get zero-row frames.

CORE V2 · 90 SHORTS (1/2)

SEASONS & LEAGUE

League: `league_root season_pointer seasons season_info season_types season_type standings_core leaders_core league_notes talentpicks tournaments positions position awards award`

Season: `season_group season_groups season_group_teams season_group_children season_type_leaders season_type_corrections season_weeks season_week season_week_games season_week_powerindex season_teams season_team season_players season_coaches season_draft season_draft_round_picks season_futures season_freeagents season_powerindex season_powerindex_leaders season_awards`

Teams & venues: `teams_core team_core venues venue franchises franchise coach coach_record coach_season`

RENAMED IN 4.0.0

3.x short	4.0.0 short
`athlete_*`	`player_*`
`event`, `events`	`game`, `games`
`event_competitor*`	`game_team*`
`event_competition`	`game_competition`
`athlete_stats` (web v3)	`player_stats_v3`

Old names stay callable and warn once. Full map: js.sportsdataverse.org → Deprecated names.

CORE V2 · 90 SHORTS (2/2)

GAMES & PLAYERS

Games: `games game game_competition game_teams game_team game_team_roster game_team_linescores game_team_statistics game_team_record game_team_leaders game_odds game_probabilities game_plays game_play game_play_personnel game_situation game_status game_officials game_official_detail game_broadcasts game_predictor game_powerindex game_propbets game_leaders game_scoringplays`

Players: `players_index player_core player_career_stats player_statisticslog player_eventlog player_contracts player_awards player_seasons player_records player_injuries player_notes player_vs_player`

```
await sdv.nhl.espnNhlGameOdds(
  { event_id: 401559593 });
await sdv.nba.espnNbaPlayerVsPlayer(
  { athlete_id: 1966, opp_id: 110 });
```

Parameter names still follow ESPN's URLs (`event_id`, `athlete_id`). Core v2 payloads are `$ref`-linked; the generic parsers (`parse_items`, single-entity) tidy list and resource shapes. ESPN Core v2 403s under aggressive parallel load: keep concurrency low.

Native APIs, providers & loaders :: 1,060 wrappers · 27 families · 323 loaders

All take `{ parsed: true }` · names are sportsdataverse-py's

NATIVE · ON THE LEAGUE NAMESPACE

14 LEAGUE FAMILIES

Family (n)	Where · auth
`mlb` 78 · `mlb_statcast` 39	`sdv.mlb` · none
`nhl_api_web` 27 · `nhl_edge` 35 · `nhl_stats_rest` 21 · `nhl_records` 44	`sdv.nhl` · none
`nfl_api` 15	`sdv.nfl` · token minted for you
`pff_api` 68	`sdv.nfl` · your API key
`nfl_pro` 16	`sdv.nfl` · NFL+ token or login
`nba_stats` 128 · `wnba_stats` 111	`sdv.nba`, `sdv.wnba` · `impit`
`kenpom` 30	`sdv.mbb` · subscription
`mls_api` 12 · `nwsl_api` 9	`sdv.mls`, `sdv.nwsl`

```
await sdv.mlb.mlbSchedule({ sport_id: 1,
  date: "2024-07-04", parsed: true });
await sdv.nhl.nhlWebPbp(
  { game_id: 2023030417, parsed: true });
await sdv.nfl.nflWeeklyGameDetails(
  { season: 2024, week: 1, parsed: true });
await sdv.nba
  .nbaStatsLeaguedashplayerstats(
    { parsed: true });
```

STATS.NBA.COM / STATS.WNBA.COM

Needs the `impit` peer (browser TLS) and hangs on datacenter IPs: run from a residential connection. `league_id` `"00"` NBA, `"20"` G League, `"15"` Summer League. A missing `season` defaults to the latest season with data; `section` picks one result set.

STATCAST SEARCH

```
await sdv.mlb.mlbStatcastSearch(
  { season: 2024, player_type: "batter" });
```

Date-chunked (≤ 25k rows / chunk) CSV → typed rows; also `_minors`, `_wbc`, `mlbStatcastPlayer`.

SUBSCRIPTIONS

```
configure({ auth: { pff_api:
  bearerAuth(process.env.PFF_API_KEY) } });
await sdv.nfl.pffApiTeamStats(
  { parsed: true });
```

KenPom: `kenpomLogin`, `hasKenpomLogin`. NFL Pro: `nflProBrowserLogin` (`playwright`). PFF: `resolvePffApiKey`.

PRODUCERS & MODELS

`sdv.nba.espnNbaPbp(gameId)`, `helperNbaPlayerBox` / `TeamBox`: one ESPN summary in, hoopR / wehoop release rows out (NBA, WNBA, MBB, WBB).
`sdv.cricket.cricketWinProbability` / `cricketWpa`: offline in-play WP.

PROVIDERS · OWN NAMESPACES

13 PROVIDER FAMILIES

Namespace (n)	Auth
`sdv.odds` The Odds API (10)	your `api_key`
`sdv.sports247` 247Sports (12 + 35)	guest JWT minted; `impit`
`sdv.on3` On3 (78)	none
`sdv.cbs` CBS Sports (82)	none
`sdv.fox` Fox Sports (38)	public apikey, defaulted
`sdv.yahoo` Yahoo (107 + 2)	none
`sdv.hockeytech` HockeyTech (16 + analytics)	per-league public keys
`sdv.torvik` T-Rank (5 + `bart_wbb` 1)	none
`sdv.asa` American Soccer Analysis (15)	none

```
await sdv.odds.oddsApiSportsOdds({
  sport: "americanfootball_nfl",
  api_key: process.env.ODDS_API_KEY,
  parsed: true });
await sdv.fox.foxApiScoreboard(
  { parsed: true });
```

ODDS MATH

`sdv.odds` also ports sdv-py `wexp.market`: `devigShin`, `devigMultiplicative`, `probFromAmerican`, `spreadToProb`, `moneylinePairProb`, `logitBlend`.

HOCKEYTECH · 20 LEAGUES

`pwhl` `ahl` `ohl` `whl` `qmjhl` `echl` `sphl` `chl` `ushl` `bchl` `ajhl` `sjhl` `ojhl` `cchl` `gojhl` `mhl` `nojhl` `vijhl` `kijhl` `mjhl`. Each league gets `<lg>Pbp`, `<lg>GameShifts`, `<lg>PlayerToi`, `<lg>GameCorsi`.

```
await sdv.hockeytech.hockeytechSchedule(
  { league: "pwhl", season_id: 8,
    parsed: true });
await sdv.hockeytech.pwhlGameCorsi(42);
```

Seasons: `hockeytechSeasonId`, `mostRecentHockeytechSeason`. Key override: env `SDV_<LEAGUE>_API_KEY`.

DATASET LOADERS (323)

```
await sdv.nfl.loadNflPbp({ seasons: 2024,
  format: "columns" });  // ~4x lighter
await sdv.cfb.loadCfbRatings(
  { seasons: 2024 });
```

One per sdv-py release: `cfb` 71 · `nba` 41 · `mbb` `wbb` `wnba` 34 · `mlb` 32 · `nfl` 29 · `nhl` 27 · `pwhl` 21. Typed `Load<Name>Row`; `columns` projection; seasons below a floor throw `SeasonNotFoundError`; `maxCells` guard. Node only.

LINKS

npm: **npmjs.com/package/sportsdataverse** · GitHub: **github.com/sportsdataverse/sportsdataverse-js** · docs, tutorials, playground: **js.sportsdataverse.org** · Python: **py.sportsdataverse.org**.

sportsdataverse :: CHEAT SHEET

The SportsDataverse's Node.js / TypeScript client (npm `sportsdataverse`) — sister to **sportsdataverse-py** and the R packages (hoopR, wehoop, cfbfastR, fastRhockey, baseballr). The cross-league ESPN client (126 endpoints × 30 leagues), 1,060 native / provider wrappers in 27 families, and 323 release-dataset loaders, with a tidy `{ parsed: true }` layer and one error vocabulary. **Names are sportsdataverse-py's.** Page 1 of 4: install, namespaces, parsers, errors.

v4.0.0

Node ≥ 20.18.1 · ESM · TypeScript
js.sportsdataverse.org

GET STARTED

```
npm install sportsdataverse
```

ESM-only, ships `.d.ts` typings. Optional peers: `impit` (browser TLS: stats.nba.com, 247Sports, KenPom) and `playwright` (NFL Pro login).

```
import sdv from "sportsdataverse";  // ESM
// CommonJS: require() on Node 20.19+ /
// 22.12+, or a dynamic import
const sdv = (await import(
  "sportsdataverse")).default;
```

QUICK START

```
const board = await sdv.nba
  .espnNbaScoreboard({ parsed: true });
```

```
const box = await sdv.nba.espnNbaSummary({
  event_id: 401584793, parsed: true,
  section: "boxscore_player" });
```

```
const pbp = await sdv.cfb.loadCfbPbp({
  seasons: [2024],
  columns: ["game_id", "text", "EPA"] });
```

Every wrapper is `async` and takes one params object; snake_case and camelCase keys are interchangeable: `{ team_id: 12 }` ≡ `{ teamId: 12 }`.

V4 NAMES = SDV-PY'S

An ESPN *athlete* is a *player*, an *event* a *game*, an *event_competitor* a *game_team*: `espnNbaPlayerGamelog`, `espnNflGameTeamRoster`. Every pre-v4 name still works and warns once (`SDV_DEPRECATED_NAME`). Each wrapper answers to snake (`espn_nfl_team_roster`) and camel (`espnNflTeamRoster`).

NAMESPACE LAYOUT

Namespace	Contents
`sdv.nba .wnba .mbb .wbb .cfb .nfl .mlb .nhl`	ESPN wrappers + native APIs + `load*` + legacy `get*` (page 2)
`sdv.nbagl .mch .wch .college_baseball .college_softball .ufl .xfl .cfl`	ESPN-only leagues
`sdv.soccer` + `epl laliga bundesliga seriea ligue1 mls ligamx ucl uel nwsl wwc wc`	league-param catch-all + 12 curated aliases (`mls`, `nwsl` also carry their stats APIs)
`sdv.cricket`	ESPN cricket + in-play win probability
`sdv.hockeytech .pwhl`	20 HockeyTech leagues · PWHL loaders
`sdv.odds .sports247 .on3 .cbs .fox .yahoo .torvik .asa`	provider namespaces (page 4)
`sdv.ncaa` · `sdv.tennis` · `sdv.recruiting`	legacy / deprecated services (page 2)

43 namespaces on the default export; `await functionCount("nba")` → 348.

MULTI-LEAGUE SPORTS

```
await sdv.soccer.espnSoccerScoreboard(
  { league: "eng.1" });  // Premier League
await sdv.laliga.espnLaligaScoreboard({});
```

`soccer` / `cricket` take a `league` slug; the curated aliases pin it for you.

DISCOVERY

```
await listFunctions("nba",
  { search: "roster" });
await findTeam("Celtics", "nba");
```

Also `functionCount`, `findAthlete(name, league)`, `findEvent(date, league, { home, away })` (snake twins too).

TIDY ROWS: { PARSED: TRUE }

Every wrapper returns the **raw** payload by default (`unknown` in TypeScript). `{ parsed: true }` runs the registered parser → flat, snake_cased row objects (py's `return_parsed=True`), typed (`MlbTeamRosterRow[]`, …) where the parser-parity harness verified the endpoint on a real capture.

```
const rows: MlbTeamRosterRow[] =
  await sdv.mlb.mlbTeamRoster(
    { team_id: 147, parsed: true });
```

Summary dispatcher (21 sections)

Sections: `boxscore_player boxscore_team plays winprobability leaders game_info officials header season_series against_the_spread standings broadcasts format pickcenter odds article injuries news drives drive_plays scoring_plays`. Omit `section` → all 21.

IDS ARE STRINGS

Integer id columns are decimal strings on every surface (`"401584793"`, exact past 2^{53}): compare and join ids as strings.

BROWSER-SAFE PARSERS

```
import { parseEndpoint }
  from "sportsdataverse/parsers";
```

No node-only HTTP deps; powers the docs playground. Also `normalize`, `PARSERS`, `parserFor`.

ERRORS (ALL `SDVERROR`)

Error	Means
`NoDataError`	fetch worked, nothing there (404, ESPN `{code: 404}`)
`AssetFetchError`	fetch failed (403 / 429 / 5xx after retries, network, empty body): answer unknown
`InvalidParameterError`	400 / 422; never retried
`SeasonNotFoundError`	season outside the source
`TransportUnavailableError`	optional peer missing

A failed fetch never comes back as empty data. An empty payload parses to `[]`.

CONFIGURE, TRANSPORT, AUTH

```
import { configure, bearerAuth }
  from "sportsdataverse";
configure({ retries: 5, timeoutMs: 60_000,
  auth: { pff_api: bearerAuth(KEY) } });
```

Per family (`site_v2`, `mlb`, `nba_stats`, …) with a `default: axiosTransport`, `createImpersonatingTransport`; auth `bearerAuth headerAuth queryAuth tokenAuth sessionAuth`; `getConfig` / `resetConfig`.

PIPE WITH TIDY.JS

```
import { tidy } from "sportsdataverse";
tidy.tidy(rows, tidy.groupBy("team",
  tidy.summarize({ n: tidy.n() })));
```

ECOSYSTEM

Python: **sportsdataverse-py** (same names). R: **hoopR wehoop cfbfastR fastRhockey baseballr**. Plots: **@sportsdataverse/sdvplot**. Docs, tutorials, playground, llms.txt: **js.sportsdataverse.org**.

Legacy services :: sdv.<league>.get*

Pre-3.0 hand-written methods, kept for back-compat. Since 4.0.0 they fetch through the shared request layer (https, retries, timeout, typed errors); new code should prefer the generated `espn_*` wrappers (page 3)

8 LEAGUE SERVICES · COMMON SHAPE

GAME ENDPOINTS

On `sdv.cfb .mbb .mlb .nba .nfl .nhl .wbb .wnba` — all take an ESPN game `id`:

Function	Returns / notes
`getPlayByPlay(id)`	teams, plays, competitions, season, boxScore subset
`getBoxScore(id)`	gamepackage boxscore (+ `id`)
`getSummary(id)`	full ESPN summary subset
`getPicks(id)`	pickcenter; cfb mbb mlb nba nfl nhl only

```
const pbp = await sdv.wnba
  .getPlayByPlay(401244185);
const box = await sdv.cfb
  .getBoxScore(401628334);
```

Source: `cdn.espn.com/core/<sport>/…` game-package feeds. `getPlayByPlay` returns `{teams, id, plays, competitions, season, boxScore, seasonSeries, standings}`.

TEAMS

Function	Args
`getTeamList()`	colleges: `{group}` (cfb 80, mbb/wbb 50)
`getTeamInfo(id)`	nfl: `{id}` object
`getTeamPlayers(id)`	roster; nfl: `{id}`

```
await sdv.mbb.getTeamList({ group: 50 });
await sdv.nba.getTeamInfo(13);
await sdv.nfl.getTeamPlayers({ id: 12 });
```

Default groups: 80 = FBS (cfb) · 50 = NCAA D1 (mbb / wbb).

SCHEDULES & STANDINGS

Function	Args
`getSchedule`	`{year, month, day}` + colleges: `groups, seasontype`
`getScoreboard`	same + `limit` (300–1000)
`getStandings`	`{year}` (defaults to now)
`getConferences`	`{year}`; cfb mbb wbb only
`getRankings`	`{year, week}`; cfb only
`getWeeklySchedule`	`{week, year, seasonType}`; nfl only

```
await sdv.cfb.getSchedule(
  { year: 2024, month: 9, day: 7,
    groups: 80, seasontype: 2 });
await sdv.nfl.getWeeklySchedule(
  { week: 1, year: 2024, seasonType: 2 });
await sdv.mbb.getStandings({ year: 2024 });
await sdv.wbb.getConferences({ year: 2024 });
await sdv.cfb.getRankings(
  { year: 2024, week: 5 });
```

AVAILABILITY MATRIX

Namespace	Method count
`sdv.cfb`	15 (+ rankings, recruiting)
`sdv.mbb`	14 (+ recruiting)
`sdv.nfl`	11 (+ weekly schedule)
`sdv.mlb sdv.nba sdv.nhl`	10 each
`sdv.wbb`	10 (conferences, no picks)
`sdv.wnba`	9 (no picks / conferences)

RECRUITING SCRAPERS · CFB + MBB

247SPORTS (LEGACY SCRAPE)

Function	Args
`getPlayerRankings`	`{year, page, group, position, state}` (+ cfb `rankingsType`)
`getSchoolRankings`	`(year, page = 1)`
`getSchoolCommits`	`(school, year)`

```
await sdv.cfb.getPlayerRankings(
  { year: 2025, position: "QB" });
await sdv.mbb.getSchoolCommits(
  "Duke", 2025);
```

HTML scrape via cheerio. Superseded by `sdv.sports247` (page 4); prefer it for new code.

TENNIS

```
await sdv.tennis.getScoreboard(
  { league: "atp", year: 2024,
    month: 7, day: 14 });
```

`league`: `"atp"` or `"wta"` (ESPN site v2 scoreboard).

MODERN REPLACEMENT

```
await sdv.sports247
  .sports247InstitutionRankings({
    sport_key: 1, year: 2025,
    parsed: true });
```

`sdv.sports247` (4.0.0): 12 API + 35 site-page wrappers; the free guest JWT is minted for you; needs `impit`. `sdv.recruiting` (25, caller JWT) is **deprecated**.

NCAA.COM · SDV.NCAA

GAMES (DATA.NCAA.COM)

Function	Purpose
`getInfo(game)`	gameInfo.json
`getBoxScore(game)`	boxscore.json
`getPlayByPlay(game)`	pbp.json
`getScoreboard({...})`	`{sport, division, year, month, day}`
`getRedirectUrl(url)`	resolve an ncaa.com game URL → casablanca id

```
const sb = await sdv.ncaa.getScoreboard({
  sport: "basketball-men", division: "d1",
  year: 2024, month: 3, day: 21 });
const url = sb.games[0].game.url;
const gid = await sdv.ncaa
  .getRedirectUrl(url);
const pbp = await sdv.ncaa.getPlayByPlay(gid);
```

STATS.NCAA.ORG RANKINGS

Function	Args
`getSports()`	—
`getSeasons(sport)`	sport code
`getDivisions(sport, season)`	—
`getSportDivisionData`	`(sport, season, division, type, gameHigh)`
`getPlayerData`	`(sport, season, division, rankingPeriod, gameHigh, category)`
`getTeamData`	same args as player

Every NCAA sport code (wvb, mih, wla, wsb, …). **Deprecated in 4.0.0**: these six stats.ncaa.org scrapers; stats.ncaa.org data lives in sportsdataverse-py and the hoopR / wehoop releases (`load*`).

ESPN cross-league :: 126 endpoints × 30 leagues

Same names as **sportsdataverse-py**'s `espn_*` wrappers · keyless, no auth

PATTERN & SCOPES

One core, parameterized on ESPN `(sport, league)` slugs. Each league exposes `espn_<prefix>_<short>` / `espn<Prefix><Short>` for every short name in its scope set:

Scope	Leagues	Shorts
`universal`	all 30	117
`ncaa`	college leagues	+6
`football`	nfl · cfb	+2
`mlb`	mlb	+1

ncaa: `rankings season_recruits recruiting_years recruiting_players recruiting_rankings season_week_rankings` · football: `season_qbr season_qbr_week` · mlb: `player_hotzones`. CFB renames: `futures groups team_powerindex recruits week_rankings`.

```
await sdv.cfb.espnCfbRankings({});
await sdv.wnba.espnWnbaStandings(
  { season: 2024 });
await sdv.nfl.espnNflScoreboard(
  { week: 1, season_type: 2 });
```

ESPN-ONLY LEAGUES

```
await sdv.mch.espnMchScoreboard({});
await sdv.college_softball
  .espnCollegeSoftballScoreboard({});
await sdv.nbagl.espnNbaglStandings({});
```

LEAGUES (LEAGUES.YAML)

Sport	Prefixes
basketball	`nba wnba mbb wbb nbagl`
football	`nfl cfb ufl xfl cfl`
baseball	`mlb college_baseball college_softball`
hockey	`nhl mch wch`
soccer	`soccer` + 12 aliases
cricket	`cricket`

SITE V2 · 25 SHORTS

SCORES, TEAMS, NEWS

League: `scoreboard summary calendar news injuries transactions conferences statistics_league draft standings rankings`

Teams: `teams_site team team_roster team_schedule team_record team_depthcharts team_injuries team_transactions team_history team_news team_leaders`

Players: `player_info player_bio player_news`

WEB V3 · 5 · FITT V3 · 1

PLAYER STATS & FPI

`player_overview player_stats_v3 player_gamelog player_splits leaders` · `fpi` (ESPN Football / Basketball Power Index)

```
await sdv.nba.espnNbaPlayerGamelog(
  { athlete_id: 1966, season: 2024 });
await sdv.cfb.espnCfbFpi({ season: 2024 });
```

CDN.ESPN.COM · 5 SHORTS

GAME-PAGE DATA

`cdn_playbyplay cdn_boxscore cdn_schedule cdn_scoreboard cdn_rankings`: the espn.com page JSON, on the leagues that serve it.

SUMMARY SECTIONS

```
await sdv.cfb.espnCfbSummary({
  event_id: 401628334, parsed: true,
  section: "drive_plays" });
```

`drives drive_plays scoring_plays` populate for NFL / CFB; other sports get zero-row frames.

CORE V2 · 90 SHORTS (1/2)

SEASONS & LEAGUE

League: `league_root season_pointer seasons season_info season_types season_type standings_core leaders_core league_notes talentpicks tournaments positions position awards award`

Season: `season_group season_groups season_group_teams season_group_children season_type_leaders season_type_corrections season_weeks season_week season_week_games season_week_powerindex season_teams season_team season_players season_coaches season_draft season_draft_round_picks season_futures season_freeagents season_powerindex season_powerindex_leaders season_awards`

Teams & venues: `teams_core team_core venues venue franchises franchise coach coach_record coach_season`

RENAMED IN 4.0.0

3.x short	4.0.0 short
`athlete_*`	`player_*`
`event`, `events`	`game`, `games`
`event_competitor*`	`game_team*`
`event_competition`	`game_competition`
`athlete_stats` (web v3)	`player_stats_v3`

Old names stay callable and warn once. Full map: js.sportsdataverse.org → Deprecated names.

CORE V2 · 90 SHORTS (2/2)

GAMES & PLAYERS

Games: `games game game_competition game_teams game_team game_team_roster game_team_linescores game_team_statistics game_team_record game_team_leaders game_odds game_probabilities game_plays game_play game_play_personnel game_situation game_status game_officials game_official_detail game_broadcasts game_predictor game_powerindex game_propbets game_leaders game_scoringplays`

Players: `players_index player_core player_career_stats player_statisticslog player_eventlog player_contracts player_awards player_seasons player_records player_injuries player_notes player_vs_player`

```
await sdv.nhl.espnNhlGameOdds(
  { event_id: 401559593 });
await sdv.nba.espnNbaPlayerVsPlayer(
  { athlete_id: 1966, opp_id: 110 });
```

Parameter names still follow ESPN's URLs (`event_id`, `athlete_id`). Core v2 payloads are `$ref`-linked; the generic parsers (`parse_items`, single-entity) tidy list and resource shapes. ESPN Core v2 403s under aggressive parallel load: keep concurrency low.

Native APIs, providers & loaders :: 1,060 wrappers · 27 families · 323 loaders

All take `{ parsed: true }` · names are **sportsdataverse-py**'s

NATIVE · ON THE LEAGUE NAMESPACE

14 LEAGUE FAMILIES

Family (n)	Where · auth
`mlb` 78 · `mlb_statcast` 39	`sdv.mlb` · none
`nhl_api_web` 27 · `nhl_edge` 35 · `nhl_stats_rest` 21 · `nhl_records` 44	`sdv.nhl` · none
`nfl_api` 15	`sdv.nfl` · token minted for you
`pff_api` 68	`sdv.nfl` · your API key
`nfl_pro` 16	`sdv.nfl` · NFL+ token or login
`nba_stats` 128 · `wnba_stats` 111	`sdv.nba`, `sdv.wnba` · `impit`
`kenpom` 30	`sdv.mbb` · subscription
`mls_api` 12 · `nwsl_api` 9	`sdv.mls`, `sdv.nwsl`

```
await sdv.mlb.mlbSchedule({ sport_id: 1,
  date: "2024-07-04", parsed: true });
await sdv.nhl.nhlWebPbp(
  { game_id: 2023030417, parsed: true });
await sdv.nfl.nflWeeklyGameDetails(
  { season: 2024, week: 1, parsed: true });
await sdv.nba
  .nbaStatsLeaguedashplayerstats(
    { parsed: true });
```

STATS.NBA.COM / STATS.WNBA.COM

Needs the `impit` peer (browser TLS) and hangs on datacenter IPs: run from a residential connection. `league_id` `"00"` NBA, `"20"` G League, `"15"` Summer League. A missing `season` defaults to the latest season with data; `section` picks one result set.

STATCAST SEARCH

```
await sdv.mlb.mlbStatcastSearch(
  { season: 2024, player_type: "batter" });
```

Date-chunked (≤ 25k rows / chunk) CSV → typed rows; also `_minors`, `_wbc`, `mlbStatcastPlayer`.

SUBSCRIPTIONS

```
configure({ auth: { pff_api:
  bearerAuth(process.env.PFF_API_KEY) } });
await sdv.nfl.pffApiTeamStats(
  { parsed: true });
```

KenPom: `kenpomLogin`, `hasKenpomLogin`. NFL Pro: `nflProBrowserLogin` (`playwright`). PFF: `resolvePffApiKey`.

PRODUCERS & MODELS

`sdv.nba.espnNbaPbp(gameId)`, `helperNbaPlayerBox` / `TeamBox`: one ESPN summary in, hoopR / wehoop release rows out (NBA, WNBA, MBB, WBB).

`sdv.cricket.cricketWinProbability` / `cricketWpa`: offline in-play WP.

PROVIDERS · OWN NAMESPACES

13 PROVIDER FAMILIES

Namespace (n)	Auth
`sdv.odds` The Odds API (10)	your `api_key`
`sdv.sports247` 247Sports (12 + 35)	guest JWT minted; `impit`
`sdv.on3` On3 (78)	none
`sdv.cbs` CBS Sports (82)	none
`sdv.fox` Fox Sports (38)	public apikey, defaulted
`sdv.yahoo` Yahoo (107 + 2)	none
`sdv.hockeytech` HockeyTech (16 + analytics)	per-league public keys
`sdv.torvik` T-Rank (5 + `bart_wbb` 1)	none
`sdv.asa` American Soccer Analysis (15)	none

```
await sdv.odds.oddsApiSportsOdds({
  sport: "americanfootball_nfl",
  api_key: process.env.ODDS_API_KEY,
  parsed: true });
await sdv.fox.foxApiScoreboard(
  { parsed: true });
```

ODDS MATH

`sdv.odds` also ports sdv-py `wexp.market`: `devigShin`, `devigMultiplicative`, `probFromAmerican`, `spreadToProb`, `moneylinePairProb`, `logitBlend`.

HOCKEYTECH · 20 LEAGUES

`pwhl` `ahl` `ohl` `whl` `qmjhl` `echl` `sphl` `chl` `ushl` `bchl` `ajhl` `sjhl` `ojhl` `cchl` `gojhl` `mhl` `nojhl` `vijhl` `kijhl` `mjhl`. Each league gets `<lg>Pbp`, `<lg>GameShifts`, `<lg>PlayerToi`, `<lg>GameCorsi`.

```
await sdv.hockeytech.hockeytechSchedule(
  { league: "pwhl", season_id: 8,
    parsed: true });
await sdv.hockeytech.pwhlGameCorsi(42);
```

Seasons: `hockeytechSeasonId`, `mostRecentHockeytechSeason`. Key override: env `SDV_<LEAGUE>_API_KEY`.

DATASET LOADERS (323)

```
await sdv.nfl.loadNflPbp({ seasons: 2024,
  format: "columns" });  // ~4x lighter
await sdv.cfb.loadCfbRatings(
  { seasons: 2024 });
```

One per sdv-py release: `cfb` 71 · `nba` 41 · `mbb` `wbb` `wnba` 34 · `mlb` 32 · `nfl` 29 · `nhl` 27 · `pwhl` 21. Typed `Load<Name>Row`; `columns` projection; seasons below a floor throw `SeasonNotFoundError`; `maxCells` guard. Node only.

LINKS

npm: **npmjs.com/package/sportsdataverse** · GitHub: **github.com/sportsdataverse/sportsdataverse-js** · docs, tutorials, playground: **js.sportsdataverse.org** · Python: **py.sportsdataverse.org**.